

ISLAMIC UNIVERSITY OF TECHNOLOGY

Board Bazar, Gazipur, Dhaka.



Artificial Intelligence Lab: CSE 4618 **Lab 5 - Markov Decision Process**

Written by - Namisa Najah Raisa
Student ID: 210042112

Sixth Semester
Department of Computer Science and Engineering
BSc. in Software Engineering

August 29, 2025

Contents

1	Introduction	3
2	Question 1: Value Iteration	3
2.1	Problem Analysis	3
2.2	Solution	3
2.3	Findings	6
3	Question 2: Bridge Crossing Analysis	7
3.1	Problem Analysis	7
3.2	Solution	7
3.3	Findings	8
4	Question 3: Policies	8
4.1	Problem Analysis	8
4.2	Solutions	9
4.2.1	3(a) - Prefer the close exit (+1), risking the cliff (-10)	9
4.2.2	3(b) - Prefer the close exit (+1), avoiding the cliff (-1)	9
4.2.3	3(c) - Prefer the distant exit (+10), risking the cliff (-10)	10
4.2.4	3(d) - Prefer the distant exit (+10), avoiding the cliff (-10)	10
4.2.5	3(e) - Avoid both exits and the cliff (so an episode should never terminate)	11
4.3	Findings	11
5	Question 4: Asynchronous Value Iteration	11
5.1	Problem Analysis	11
5.2	Solution	11
5.3	Findings	13
6	Challenges	14

1 Introduction

In this lab, our task was to explore Markov Decision Processes (MDPs). We implemented algorithms such as value iteration and asynchronous value iteration compute optimal policies. The goal was to understand how agents can maximize long-term rewards by balancing immediate and future outcomes in probabilistic environments.

2 Question 1: Value Iteration

2.1 Problem Analysis

The task was to implement a **ValueIterationAgent** that, given an MDP, runs **batch** value iteration for a specified number of iterations during construction and then returns the synthesized policy. Specifically, we had to implement the Bellman equation:

$$V_{k+1}(s) = \max_a \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma V_k(s') \right],$$

where the values from the previous iteration are used. In addition, we needed to implement the methods **computeQValueFromValues(state, action)** and **computeActionFromValues(state)**, where terminal states return **None**. The agent was required to use the MDP API (**getStates**, **getPossibleActions**, **getTransitionStatesAndProbs**, **getReward**) while using the given discount factor γ and iteration count.

2.2 Solution

- I initialized the **ValueIterationAgent** with an MDP, a discount factor, and a number of iterations. I created **self.values** to store state values and immediately called **runValueIteration()** to compute state-value estimates.
- In **runValueIteration()**, I performed the main value iteration loop. For each iteration, I created a fresh **new_values** container. For each state:
 - If the state was terminal, I set its value to 0.
 - For non-terminal states, I computed Q-values for all possible actions using **computeQValueFromValues()** and updated the state value to the maximum Q-value.

At the end of the iteration, I set **self.values = new_values** to ensure all updates used values from the previous iteration.

- In **computeQValueFromValues(state, action)**, I calculated the Q-value for a state-action pair by iterating over all possible next states and their transition probabilities, summing the expected rewards plus discounted future values.

- In `computeActionFromValues(state)`, I selected the best action based on the computed values. I returned `None` for terminal states. For non-terminal states, I computed Q-values for all legal actions and chose the action with the highest Q-value. I used this function for `getPolicy()` and `getAction()`.
- I also implemented `getValue(state)` and `getQValue(state, action)` to allow querying of the computed state values and Q-values.
- Overall, I first computed the value estimates through repeated Bellman updates, then used Q-values to determine the optimal actions, producing a complete policy for the MDP.

```
class ValueIterationAgent(ValueEstimationAgent):
    """
    * Please read LearningAgents.py before reading this.*

    A ValueIterationAgent takes a Markov decision process
    (see mdp.py) on initialization and runs value iteration
    for a given number of iterations using the supplied
    discount factor.
    """
    def __init__(self, mdp, discount = 0.9, iterations = 100):
        """
        Your value iteration agent should take an mdp on
        construction, run the indicated number of iterations
        and then act according to the resulting policy.

        Some useful mdp methods you will use:
            mdp.getStates()
            mdp.getPossibleActions(state)
            mdp.getTransitionStatesAndProbs(state, action)
            mdp.getReward(state, action, nextState)
            mdp.isTerminal(state)
        """
        self.mdp = mdp
        self.discount = discount
        self.iterations = iterations
        self.values = util.Counter() # A Counter is a dict with default 0
        self.runValueIteration()
```

Figure 1: Initialization of problem 1

```

def runValueIteration(self):
    # Write value iteration code here
    """ YOUR CODE HERE """
    # 210042112
    states = self.mdp.getStates()
    for i in range(self.iterations):
        new_values = util.Counter()
        for state in states:
            if self.mdp.isTerminal(state):
                new_values[state] = 0
            else:
                possible_actions = self.mdp.getPossibleActions(state)
                q_values = [self.computeQValueFromValues(state, action) for action in possible_actions]
                new_values[state] = max(q_values) if q_values else 0
        self.values = new_values

def getValue(self, state):
    """
    Return the value of the state (computed in __init__).
    """
    return self.values[state]

```

Figure 2: runValueIteration function

```

def computeQValueFromValues(self, state, action):
    """
    Compute the Q-value of action in state from the
    value function stored in self.values.
    """
    """ YOUR CODE HERE """
    # 210042112
    # util.raiseNotDefined()
    q_value = 0
    for next_state, prob in self.mdp.getTransitionStatesAndProbs(state, action):
        reward = self.mdp.getReward(state, action, next_state)
        q_value += prob * (reward + self.discount * self.values[next_state])
    return q_value

```

Figure 3: computeQValueFromValues function

```

def computeActionFromValues(self, state):
    """
        The policy is the best action in the given state
        according to the values currently stored in self.values.

        You may break ties any way you see fit. Note that if
        there are no legal actions, which is the case at the
        terminal state, you should return None.
    """
    """ YOUR CODE HERE """
    # 210042112
    # util.raiseNotDefined()
    if self.mdp.isTerminal(state):
        return None

    possible_actions = self.mdp.getPossibleActions(state)
    best_action = None
    best_value = float('-inf')

    for action in possible_actions:
        q_value = self.computeQValueFromValues(state, action)
        if q_value > best_value:
            best_value = q_value
            best_action = action

    return best_action

```

Figure 4: computeActionFromValues function

2.3 Findings

Using a fresh new_values each iteration was essential to avoid mixing updated and old estimates and keep the math correct. Computing Q-values made policy extraction straightforward and naturally handled stochastic transitions, but convergence and policy quality depended strongly on the number of iterations and the discount factor.

3 Question 2: Bridge Crossing Analysis

3.1 Problem Analysis

The task was to analyze the BridgeGrid environment, where the agent starts near a low-reward terminal separated from a high-reward terminal by a narrow bridge over a high-penalty chasm. The challenge was to determine how the agent's expected state values are influenced by the discount factor and action noise. With the default settings (discount = 0.9, noise = 0.2), the optimal policy avoids the bridge, as the risk outweighs the distant reward. The goal was to adjust either the discount or noise so that the agent becomes willing to cross the bridge and pursue the higher reward.

3.2 Solution

I chose **Discount = 0.9** and **Noise = 0.0**. Setting the noise to 0 makes the actions deterministic, so the agent no longer risks accidentally falling into the high-penalty chance when crossing the bridge. With the discount still at 0.9, the agent values the future high reward enough relative to immediate rewards, so the expected value of states on the bridge becomes positive.

```
def question2():  
    # 210042112  
    answerDiscount = 0.9  
    answerNoise = 0.0  
    return answerDiscount, answerNoise
```

Figure 5: Values put for question 2



Figure 6: values after 100 iterations with discount 0.9 and noise 0.0

3.3 Findings

By reducing the value of the noise and increasing the value of the discount, it is possible to achieve the intended behavior of the agent.

4 Question 3: Policies

4.1 Problem Analysis

The task was to try out different values of three parameters - noise, discount, and the living reward in an MDP to produce different optimal policy types.

4.2 Solutions

4.2.1 3(a) - Prefer the close exit (+1), risking the cliff (-10)

```
def question3a():  
    # 210042112  
    answerDiscount = 0.1 # grabs reward quickly and avoids negative cells  
    answerNoise = 0.0  
    answerLivingReward = 0.0 # so it doesn't wander around  
    return answerDiscount, answerNoise, answerLivingReward  
    # If not possible, return 'NOT POSSIBLE'
```

Figure 7: 3(a)

By setting a low discount factor (0.1), the agent strongly favors immediate rewards over distant ones. Thus, the close exit (+1) is valued more highly than the distant exit (+10). With a living reward of 0.0, the agent is not penalized for surviving longer, so it does not avoid risky paths. As a result, the agent tends to prefer the short path toward the near exit, even though it may risk falling into the cliff (-10).

4.2.2 3(b) - Prefer the close exit (+1), avoiding the cliff (-1)

```
def question3b():  
    # 210042112  
    answerDiscount = 0.1  
    answerNoise = 0.01 # small noise to add a little bit of uncertainty  
    answerLivingReward = 0.0  
    return answerDiscount, answerNoise, answerLivingReward  
    # If not possible, return 'NOT POSSIBLE'
```

Figure 8: 3(b)

The agent prefers the close exit (+1) due to the low discount factor, which devalues the far exit (+10). However, setting a small answerNoise (0.01) makes risky paths less reliable, since the agent might slip into the cliff (-1). As a result, the agent avoids hugging the cliff edge and instead follows a safer path to the near exit.

4.2.3 3(c) - Prefer the distant exit (+10), risking the cliff (-10)

```
def question3c():  
    # 210042112  
    answerDiscount = 0.9 # far sighted  
    answerNoise = 0.0  
    answerLivingReward = 0.0  
    return answerDiscount, answerNoise, answerLivingReward  
    # If not possible, return 'NOT POSSIBLE'
```

Figure 9: 3(c)

With a high discount factor (0.9), the agent values the distant +10 exit more than the nearby +1 exit. Since there is no living penalty, the agent does not mind taking extra steps, and with zero noise, it confidently chooses the shortest path; even if that means traveling dangerously close to the cliff. This results in a policy where the agent prefers the distant exit while risking the cliff.

4.2.4 3(d) - Prefer the distant exit (+10), avoiding the cliff (-10)

```
def question3d():  
    # 210042112  
    answerDiscount = 0.99  
    answerNoise = 0.01  
    answerLivingReward = 0.0  
    return answerDiscount, answerNoise, answerLivingReward  
    # If not possible, return 'NOT POSSIBLE'
```

Figure 10: 3(d)

With a high discount factor (0.99), the agent strongly prefers the distant +10 exit over the nearby +1 exit. Introducing a small noise (0.01) makes risky moves near the cliff unreliable, so the agent chooses a safer path that avoids the cliff. Since there is no living penalty, the agent can afford the detour. Thus, the agent's policy is to go for the distant exit while avoiding the cliff.

4.2.5 3(e) - Avoid both exits and the cliff (so an episode should never terminate)

```
def question3e():  
    # 210042112  
    answerDiscount = 0.0  
    answerNoise = 0.0  
    answerLivingReward = 1.0  
    return answerDiscount, answerNoise, answerLivingReward  
    # If not possible, return 'NOT POSSIBLE'
```

Figure 11: 3(e)

The answerDiscount and answerNoise are set to 0, and answerLivingReward is set to 1 to ensure that the agent keeps moving indefinitely without reaching the goal. Because while wandering around and eating pellets, the reward can eventually reach a lot higher than +10.

4.3 Findings

Different values for different parameters enforce unique behaviors of the agent.

5 Question 4: Asynchronous Value Iteration

5.1 Problem Analysis

The task was to implement an Asynchronous Value Iteration Agent. The goal was to update one state per iteration in a cyclic order, rather than updating all states simultaneously like in standard batch value iteration. Terminal states were to be skipped, and the value of each state was updated based on the maximum expected Q-value of its possible actions.

5.2 Solution

- I implemented an **Asynchronous Value Iteration Agent** that performs value iteration by updating **only one state per iteration** in a cyclic order.
- In each iteration, I selected the state to update using `i % len(states)` to cycle through all states repeatedly.
- For terminal states, I explicitly set the value to 0, as they have no future rewards.
- For non-terminal states, I computed the **Q-values for all possible actions** from that state using `self.getQValue`.

- I then updated the value of the state to the **maximum Q-value**, representing the best expected utility of that state.
- I used **self.values in-place**, so that each updated value is immediately available for the next iteration, allowing values to propagate efficiently across the MDP.
- I repeated this process for the specified number of iterations to approximate the **optimal state values** while following the asynchronous update principle.

```
class AsynchronousValueIterationAgent(ValueIterationAgent):
    """
        * Please read learningAgents.py before reading this.*

        An AsynchronousValueIterationAgent takes a Markov decision process
        (see mdp.py) on initialization and runs cyclic value iteration
        for a given number of iterations using the supplied
        discount factor.
    """
    def __init__(self, mdp, discount = 0.9, iterations = 1000):
        """
            Your cyclic value iteration agent should take an mdp on
            construction, run the indicated number of iterations,
            and then act according to the resulting policy. Each iteration
            updates the value of only one state, which cycles through
            the states list. If the chosen state is terminal, nothing
            happens in that iteration.

            Some useful mdp methods you will use:
                mdp.getStates()
                mdp.getPossibleActions(state)
                mdp.getTransitionStatesAndProbs(state, action)
                mdp.getReward(state)
                mdp.isTerminal(state)
        """
        ValueIterationAgent.__init__(self, mdp, discount, iterations)
```

Figure 12: Asynchronous Value Iteration class initiate

```

def runValueIteration(self):
    """ YOUR CODE HERE """
    # 210042112
    states = self.mdp.getStates()
    for i in range(self.iterations):
        new_values = self.values
        state = states[i % len(states)]

        if self.mdp.isTerminal(state):
            new_values[state] = 0
        else:
            possible_actions = self.mdp.getPossibleActions(state)
            q_values = [self.getQValue(state, action) for action in possible_actions]
            new_values[state] = max(q_values) if q_values else 0

    self.values = new_values

```

Figure 13: Asynchronous Value Iteration

5.3 Findings

Asynchronous updates make value propagation more efficient, since updated values are immediately used in subsequent iterations. Unlike batch updates, this approach converges faster in practice for many MDPs.

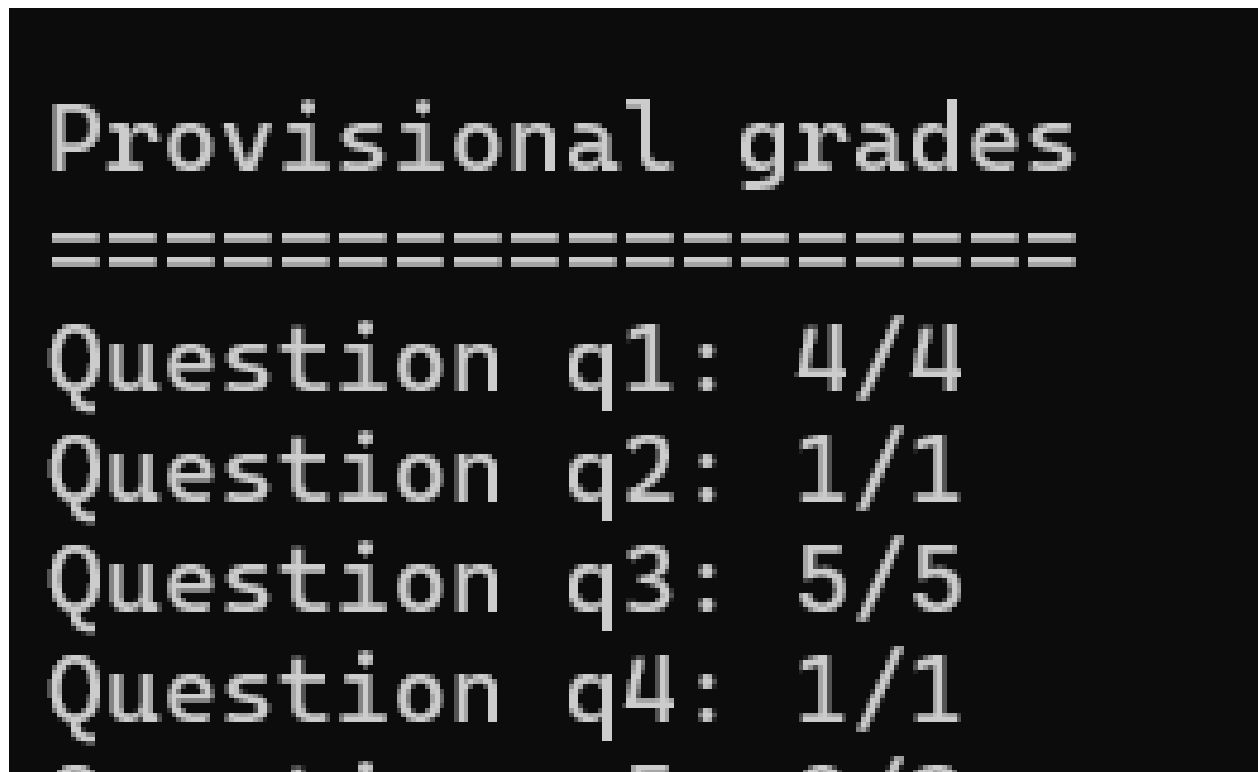


Figure 14: All grades

6 Challenges

It took me a long time to figure out how the different parameters were affecting the results in tasks 2 and 3. I kept trying out different value combinations to watch the outcome. The asynchronous task was the trickiest. Figuring out how to do the cyclic updates one state at a time wasn't straightforward, and it took me a while to understand it.